St. Francis Institute of Technology

(Engineering College)

An Autonomous Institute, Affiliated to University of Mumbai

NAAC A+ Accredited | CMPN, EXTC, INFT NBA Accredited | ISO 9001:2015 Certified

Department of Artificial Intelligence and Machine Learning

Academic Year: 2025-2026 **Term**: Even (Jan. 2026 – Jun. 2026)
Class / Branch: SE – AIML **Semester**: IV
Course: Web Programming Lab. (AI4VS_LR4)
Date of Assignment: 1 1 /03/2026 **Date of Submission:** 11/03/2026

Pre-Lab Exercises for Experiment-11

Part A: Understanding NoSQL and MongoDB

Task 1: Conceptual Preparation

Answer the following questions:

Question: What is NoSQL?
Answer: NoSQL refers to non-relational databases designed for scalable, flexible storage of unstructured or semi-structured data without fixed schemas.

Question: How is NoSQL different from SQL databases?
Answer: NoSQL uses dynamic schemas, horizontal scaling, and BASE consistency for big data/flexible models; SQL uses fixed schemas, tables, joins, and ACID transactions for structured relational data.

Question: What is MongoDB?
Answer: MongoDB is a document-oriented NoSQL database that stores data in flexible JSON-like BSON documents, optimized for scalability and high-performance queries.

Question: What is a document in MongoDB?
Answer: A document is a single record in MongoDB, represented as a BSON object with key-value pairs, similar to a JSON object but with extended data types.

Question: What is a collection?
Answer: A collection is a group of MongoDB documents, analogous to a table in SQL but without a fixed schema—documents can have varying structures.

Question: What is BSON?
Answer: BSON (Binary JSON) is a binary-encoded serialization of JSON-like documents used by MongoDB for compact storage and fast parsing of complex data types.

Question: What are advantages of MongoDB?
Answer: MongoDB offers schema flexibility, horizontal scaling via sharding, high performance for read/write-heavy apps, rich querying, and easy JSON integration.

Part B: MongoDB Practice (Without Express)

Task 2: Install and Explore MongoDB

1. Install MongoDB locally OR create a free account in **MongoDB Atlas**.
2. Create a new database named:
   - practiceDB
3. Create a collection:
   - students

Perform the following operations:

- Insert 3 student records manually.
- View inserted records.
- Delete one record.
- Update one record.

```
test> use practice-DB
switched to db practice-DB
practice-DB> db.students.insertMany([
... {name:"Nihar", age:19, course:"AIML"},
... {name:"Viraj", age:19, course:"AIML"},
... {name:"Neel", age:19, course:"AIML"}
... ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId("69ad678d9ab5fa37b82db833"),
    '1': ObjectId("69ad678d9ab5fa37b82db834"),
    '2': ObjectId("69ad678d9ab5fa37b82db835")
  }
}
practice-DB> db.students.find()
[
  {
    _id: ObjectId("69ad678d9ab5fa37b82db833"),
    name: 'Nihar',
    age: 19,
    course: 'AIML'
  },
  {
    _id: ObjectId("69ad678d9ab5fa37b82db834"),
    name: 'Viraj',
    age: 19,
    course: 'AIML'
  },
  {
    _id: ObjectId("69ad678d9ab5fa37b82db835"),
    name: 'Neel',
    age: 19,
    course: 'AIML'
  }
]
```

```
practice-DB> db.students.updateOne(
... {name:"Nihar"},
... {$set:{age:20}}
... )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
practice-DB> db.students.deleteOne({name:"Neel"})
{ acknowledged: true, deletedCount: 1 }
practice-DB> db.students.find()
[
  {
    _id: ObjectId("69ad678d9ab5fa37b82db833"),
    name: 'Nihar',
    age: 20,
    course: 'AIML'
  },
  {
    _id: ObjectId("69ad678d9ab5fa37b82db834"),
    name: 'Viraj',
    age: 19,
    course: 'AIML'
  }
]
practice-DB>
```

Submission:

Take screenshots of:

- Database creation
- Inserted documents
- Updated document

Part C: Mongoose Familiarization

Task 3: Setup Basic Backend with Mongoose

1. Create a new Node.js project.
2. Install:
   - express
   - mongoose
3. Connect to MongoDB database.
4. Confirm successful connection in terminal.

```
const express = require("express");
const mongoose = require("mongoose");

const app = express();
app.use(express.json());

mongoose.connect("mongodb://127.0.0.1:27017/practice-DB")
.then(() => console.log("MongoDB Connected"))
.catch(err => console.log(err));

const userSchema = new mongoose.Schema({
 name: String,
 email: String,
 age: Number
});

const User = mongoose.model("User", userSchema);

app.get("/", (req,res)=>{
 res.send("Server Running");
});

app.get("/users", async (req,res)=>{
 const users = await User.find();
 res.json(users);
});

app.listen(3000, ()=>{
 console.log("Server running on port 3000");
});
```

```
C:\Users\Pramod\mongoose-backend>node server.js
Server running on port 3000
MongoDB Connected
```

```
C:\Users\Pramod\mongoose-backend>node server.js
Server running on port 3000
MongoDB Connected
```

localhost:3000/adduser

User Added Successfully

localhost:3000/users

Pretty-print

[]

Questions:

Question: What happens if connection string is wrong?
Answer: MongoDB connection fails with authentication errors, timeout exceptions, or "invalid scheme" errors; app cannot connect to database and operations throw MongoNetworkError or MongoParseError.

Question: What is the purpose of mongoose?
Answer: Mongoose is an ODM library for MongoDB/Node.js that provides schema validation, model-based querying, middleware support, and easier database interactions over raw MongoDB drivers.

Task 4: Create Schema and Model

1. Create a schema with fields:
   - name
   - email
   - age
2. Create a model using the schema.
3. Insert one record using the model.
4. Retrieve all records from collection.

```
const express = require("express");
const mongoose = require("mongoose");

const app = express();
app.use(express.json());

// MongoDB connection
mongoose.connect("mongodb://127.0.0.1:27017/practice-DB")
.then(() => console.log("MongoDB Connected"))
.catch(err => console.log(err));

// Schema
const userSchema = new mongoose.Schema({
  name: String,
  email: String,
  age: Number
});

// Model
const User = mongoose.model("User", userSchema);

// Home route
app.get("/", (req,res)=>{
  res.send("Server Running");
});

// Insert user
app.get("/adduser", async (req,res)=>{

  const newUser = new User({
    name:"Nihar",
    email:"Nihar@gmail.com",
    age:19
  });

  await newUser.save();

  res.send("User Added Successfully");
});

// View users
app.get("/users", async (req,res)=>{

  const users = await User.find();

  res.json(users);

});

app.listen(3000, ()=>{
  console.log("Server running on port 3000");
});
```

localhost:3000/users

Pretty-print ☑


```
[
  {
    "_id": "69ad8af826b1df1a1e81e08e",
    "name": "Nihar",
    "email": "Nihar@gmail.com",
    "age": 19,
    "__v": 0
  }
]
```


localhost:3000/adduser

User Added Successfully

Observation:

Question: Is the data stored in MongoDB?
Answer: Yes, MongoDB stores data persistently on disk as BSON documents within collections, managed by storage engines like WiredTiger.

Question: Does schema restrict data structure?
Answer: No, MongoDB is schemaless—documents in a collection can have different fields/structures; Mongoose schemas optionally enforce validation but don't restrict raw MongoDB storage.

Part D: REST API Practice

Task 5: Create Basic REST APIs

Create the following routes:

1. POST → Add user
2. GET → Retrieve all users

Test using:

- Postman

Verify:

- Status codes
- JSON response format

server.js

File Edit View

```
const express = require("express");
const mongoose = require("mongoose");

const app = express();
app.use(express.json());

// Connect MongoDB
mongoose.connect("mongodb://127.0.0.1:27017/practice-DB")
.then(() => console.log("MongoDB Connected"))
.catch(err => console.log(err));

// Schema
const userSchema = new mongoose.Schema({
 name: String,
 email: String,
 age: Number
});

// Model
const User = mongoose.model("User", userSchema);

// POST API (Add user)
app.post("/users", async (req,res)=>{

 const user = new User(req.body);

 await user.save();

 res.send("User Added");

});

// GET API (Get all users)
app.get("/users", async (req,res)=>{

 const users = await User.find();

 res.json(users);

});

app.listen(3000, ()=>{
 console.log("Server running on port 3000");
});
```

localhost:3000/users

Pretty-print


```
[
  {
    "_id": "69ad8af826b1df1a1e81e08e",
    "name": "Nihar",
    "email": "Nihar@gmail.com",
    "age": 19,
    "__v": 0
  }
]
```



```
[
  {
    "_id": "69ad8af826b1df1a1e81e08e",
    "name": "Nihar",
    "email": "Nihar@gmail.com",
    "age": 19,
    "__v": 0
  }
```


GET http://localhost:3000/users

JSON Preview Visualize


```
{
    "_id": "69adcccbb2a52fddf117c088",
    "__v": 0
}
```


POST http://localhost:3000/users

Part E: React API Practice

Task 6: Fetch Data in React

1. Create a simple React app.
2. Create a component:
   - UserList
3. Fetch data from backend GET API.
4. Store data in state.
5. Display data using:
   - List OR Table

```react/app.js
import React from "react";
import UserList from "./components/UserList";

function App() {
  return (
    <div>
      <h1>React API Practice</h1>
      <UserList />
    </div>
  );
}

export default App;
```

```react/components/UserList.js
import React, { useEffect, useState } from "react";

function UserList() {

  const [users, setUsers] = useState([]);

  useEffect(() => {
    fetch("http://localhost:5000/users")
      .then(response => response.json())
      .then(data => setUsers(data))
      .catch(error => console.log(error));
  }, []);

  return (
    <div>
      <h2>User List</h2>

      <ul>
        {users.map((user, index) => (
          <li key={index}>
            {user.name} - {user.email}
          </li>
        ))}
      </ul>

    </div>
  );
}

export default UserList;
```

```backend/server.js
const express = require("express");
const cors = require("cors");

const app = express();
app.use(cors());

app.get("/users", (req, res) => {
  res.json([
    { name: "Nihar", email: "nihar@gmail.com", age: 20 },
    { name: "Rahul", email: "rahul@gmail.com", age: 21 }
  ]);
});

app.listen(5000, () => {
  console.log("Server running on port 5000");
});
```

Question 1: Does UI update after fetching data?

Yes, the UI updates automatically if you properly update the component's state (e.g., setData(fetchedData)) after a successful fetch in frameworks like React or Flutter. Without state changes, the UI remains static even if data arrives.

Question 2: What happens if backend is not running?

The fetch request fails with a network error, timeout, or HTTP 5xx status. Handle it in .catch() or try-catch by showing an error message like "Server unavailable" and optionally use cached data or retry logic.

Part F: Data Persistence Practice

Task 7: Restart Server Test

1. Add user using POST request.
2. Restart backend server.
3. Fetch users again.

Body

{} JSON Preview Visualize

```
{
    "message": "User added successfully",
    "users": [
        {
            "name": "Nihar",
            "email": "nihar@gmail.com",
            "age": 20
        },
        {
            "name": "Rahul",
            "email": "rahul@gmail.com",
            "age": 21
        },
        {
            "name": "Amit",
            "email": "amit@gmail.com",
            "age": 22
        }
    ]
}
```

POST http://localhost:5000/users

Docs Params Authorization Headers 8 Body Scri

raw JSON

```
{
  "name": "Amit",
  "email": "amit@gmail.com",
  "age": 22
}
```

localhost:5000/users

Pretty-print

```
[
  {
    "name": "Nihar",
    "email": "nihar@gmail.com",
    "age": 20
  },
  {
    "name": "Rahul",
    "email": "rahul@gmail.com",
    "age": 21
  }
]
```

```
const express = require("express");
const cors = require("cors");

const app = express();

app.use(cors());
app.use(express.json());

// temporary memory storage
let users = [
  { name: "Nihar", email: "nihar@gmail.com", age: 20 },
  { name: "Rahul", email: "rahul@gmail.com", age: 21 }
];

// GET API
app.get("/users", (req, res) => {
  res.json(users);
});

// POST API
app.post("/users", (req, res) => {
  const newUser = req.body;
  users.push(newUser);
  res.json({
    message: "User added successfully",
    users: users
  });
});

app.listen(5000, () => {
  console.log("Server running on port 5000");
});
```

localhost:5000/users

Pretty-print

```
[
  {
    "name": "Nihar",
    "email": "nihar@gmail.com",
    "age": 20
  },
  {
    "name": "Rahul",
    "email": "rahul@gmail.com",
    "age": 21
  },
  {
    "name": "Amit",
    "email": "amit@gmail.com",
    "age": 22
  }
]
```

Question:

Does data still exist? Why?

No, the data does not still exist after restarting the server because it is stored only in the server's temporary memory (RAM). When the backend server restarts, the memory is cleared and all previously added data is lost. To store data permanently, a database such as MongoDB or MySQL would be required.

St. Francis Institute of Technology

(Engineering College)

An Autonomous Institute, Affiliated to University of Mumbai

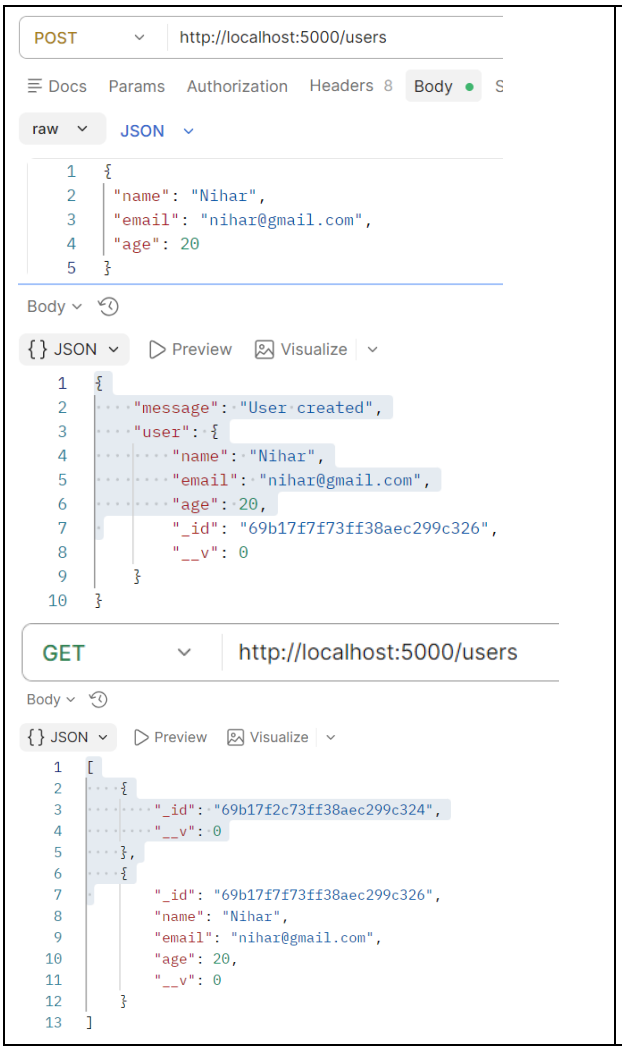
NAAC A+ Accredited | CMPN, EXTC, INFT NBA Accredited | ISO 9001:2015 Certified

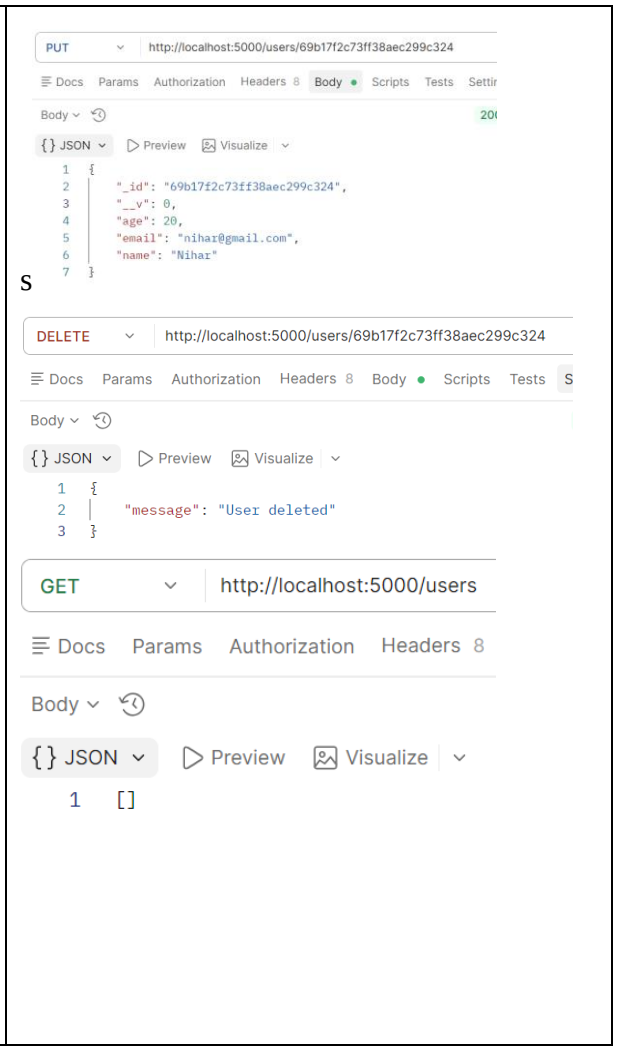
Department of Artificial Intelligence and Machine Learning

Task 8: Perform All CRUD Operations via API

Activities:

1. Extend the backend to include:
   - PUT → Update user by ID
   - DELETE → Delete user by ID
2. Test all four operations using Postman:
   - Create user (POST)
   - Retrieve users (GET)
   - Update user (PUT)
   - Delete user (DELETE)
3. Verify changes in MongoDB database.





server.js

File Edit View

```
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");

const app = express();

app.use(express.json());
app.use(cors());

mongoose.connect("mongodb://127.0.0.1:27017/userDB")
  .then(() => {
    console.log("MongoDB Connected");
  })
  .catch((err) => {
    console.log(err);
  });

const userSchema = new mongoose.Schema({
  name: String,
  email: String,
  age: Number
});

const User = mongoose.model("User", userSchema);

/* CREATE USER */
app.post("/users", async (req, res) => {
  const user = new User(req.body);
  await user.save();
  res.json({ message: "User created", user });
});

/* GET USERS */
app.get("/users", async (req, res) => {
  const users = await User.find();
  res.json(users);
});

/* UPDATE USER */
app.put("/users/:id", async (req, res) => {
  const user = await User.findByIdAndUpdate(
    req.params.id,
    req.body,
    { new: true }
  );
  res.json(user);
});

/* DELETE USER */
app.delete("/users/:id", async (req, res) => {
  await User.findByIdAndDelete(req.params.id);
  res.json({ message: "User deleted" });
});

app.listen(5000, () => {
  console.log("Server running on port 5000");
});
```

1. What status codes are returned for each operation?

- **POST (Create User)** → 200 OK or sometimes 201 Created
- **GET (Retrieve Users)** → 200 OK
- **PUT (Update User)** → 200 OK
- **DELETE (Delete User)** → 200 OK

These status codes indicate that the API request was processed successfully.

2. What happens if an invalid ID is used?

If an invalid or non-existent ID is used in the **PUT or DELETE request**, the server cannot find the user in the **MongoDB**. The API may return an error or a null response indicating that the record does not exist.

3. Does the database reflect changes immediately?

Yes. The database reflects changes immediately. When a user is created, updated, or deleted through the API tested using **Postman**, the changes are instantly stored or modified in the MongoDB database.

St. Francis Institute of Technology

(Engineering College)

An Autonomous Institute, Affiliated to University of Mumbai

NAAC A+ Accredited | CMPN, EXTC, INFT NBA Accredited | ISO 9001:2015 Certified

Department of Artificial Intelligence and Machine Learning

Task 9: Implement Basic Validation in Mongoose

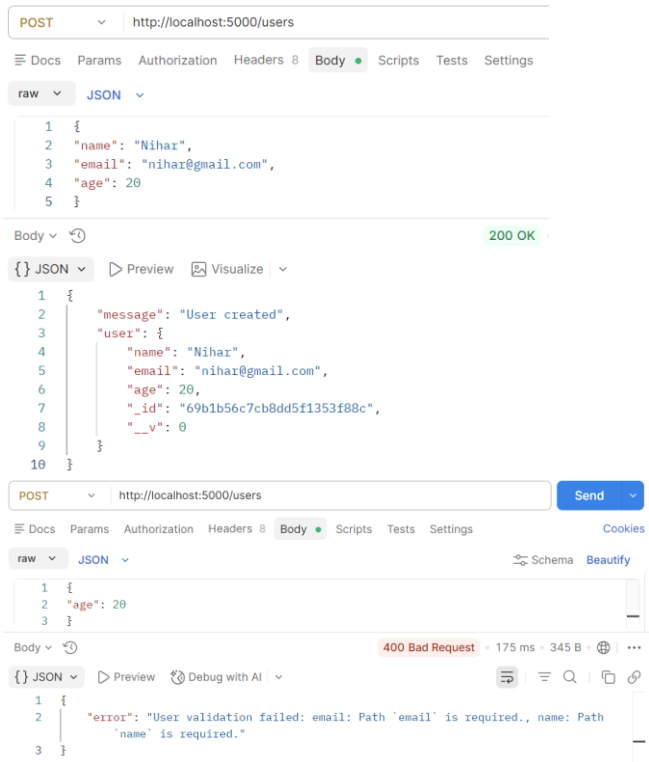
Activities:

1. Modify schema to:
   - Make name required
   - Make email required
   - Add minimum age validation
2. Try inserting:
   - Valid data
   - Invalid data (missing fields or incorrect age)
3. Observe error messages.

```
const express = require("express");
const mongoose = require("mongoose");

const app = express();
app.use(express.json());

// MongoDB connection
mongoose.connect("mongodb://127.0.0.1:27017/userDB")
.then(() => console.log("MongoDB Connected"))
.catch(err => console.log(err));

// Schema with validation
const userSchema = new mongoose.Schema({
  name: {
    type: String,
    required: true
  },
  email: {
    type: String,
    required: true
  },
  age: {
    type: Number,
    min: 18
  }
});

const User = mongoose.model("User", userSchema);

// POST API
app.post("/users", async (req, res) => {
  try {
    const user = new User(req.body);
    await user.save();
    res.json({
      message: "User created",
      user: user
    });
  } catch (error) {
    res.status(400).json({
      error: error.message
    });
  }
});

app.listen(5000, () => {
  console.log("Server running on port 5000");
});
```

3. Where are validation errors displayed?

Validation errors are displayed in the API response when the request is sent from tools like **Postman**, and they may also appear in the server console logs when the error occurs.



1. Does MongoDB allow invalid data without schema?

Yes, **MongoDB** is schema-less, which means it does not enforce a fixed structure for documents. Because of this, it allows storing any type of data even if fields are missing or contain incorrect values.

---

2. How does Mongoose validation improve data quality?

Mongoose adds validation rules such as required fields, minimum or maximum values, and data types. These rules ensure that only valid and properly structured data is saved in the database, improving data accuracy and consistency.

Task 10: Handle Errors Gracefully

Activities:

1. Add error handling in:
   - Database connection
   - POST route
   - GET route
2. Send incorrect requests from Postman:
   - Invalid ID
   - Missing fields
3. Return proper:
   - Status codes (400, 404, 500)
   - JSON error messages

```
const express = require("express");
const mongoose = require("mongoose");

const app = express();
app.use(express.json());

// Database connection with error handling
mongoose.connect("mongodb://127.0.0.1:27017/userDB")
.then(() => console.log("MongoDB Connected"))
.catch(err => console.log("Database connection error:", err.message));

// Schema
const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true },
  age: { type: Number, min: 18 }
});

const User = mongoose.model("User", userSchema);

// POST route with error handling
app.post("/users", async (req, res) => {
  try {
    const user = new User(req.body);
    await user.save();
    res.status(201).json({ message: "User created", user });
  } catch (error) {
    res.status(400).json({ error: error.message });
  }
});

// GET all users with error handling
app.get("/users", async (req, res) => {
  try {
    const users = await User.find();
    res.status(200).json(users);
  } catch (error) {
    res.status(500).json({ error: "Server error" });
  }
});

// GET user by ID with error handling
app.get("/users/:id", async (req, res) => {
  try {
    const user = await User.findById(req.params.id);

    if (!user) {
      return res.status(404).json({ error: "User not found" });
    }

    res.status(200).json(user);

  } catch (error) {
    res.status(400).json({ error: "Invalid ID format" });
  }
});

app.listen(5000, () => {
  console.log("Server running on port 5000");
});
```

POST http://localhost:5000/users

Docs Params Authorization Headers 8 Body Scripts Tests

raw JSON

```
1  {
2  "name": "Nihar",
3  "email": "Nihar@gmail.com",
4  "age": 19
5  }
```

Body 20

{} JSON Preview Visualize

```
1  {
2      "message": "User created",
3      "user": {
4          "name": "Nihar",
5          "email": "Nihar@gmail.com",
6          "age": 19,
7          "_id": "69b1c2c95462c607208b2f40",
8          "__v": 0
9      }
10 }
```

POST http://localhost:5000/users

Docs Params Authorization Headers 8 Body Scripts Tests

raw JSON

```
1  {
2  "name": "Krishna",
3  "email": "krishna@gmail.com",
4  "age": 20
5  }
```

Body 201

{} JSON Preview Visualize

```
1  {
2      "message": "User created",
3      "user": {
4          "name": "Krishna",
5          "email": "krishna@gmail.com",
6          "age": 20,
7          "_id": "69b1c3605462c607208b2f42",
8          "__v": 0
9      }
10 }
```

Observations:

1. What status code is returned for bad requests?

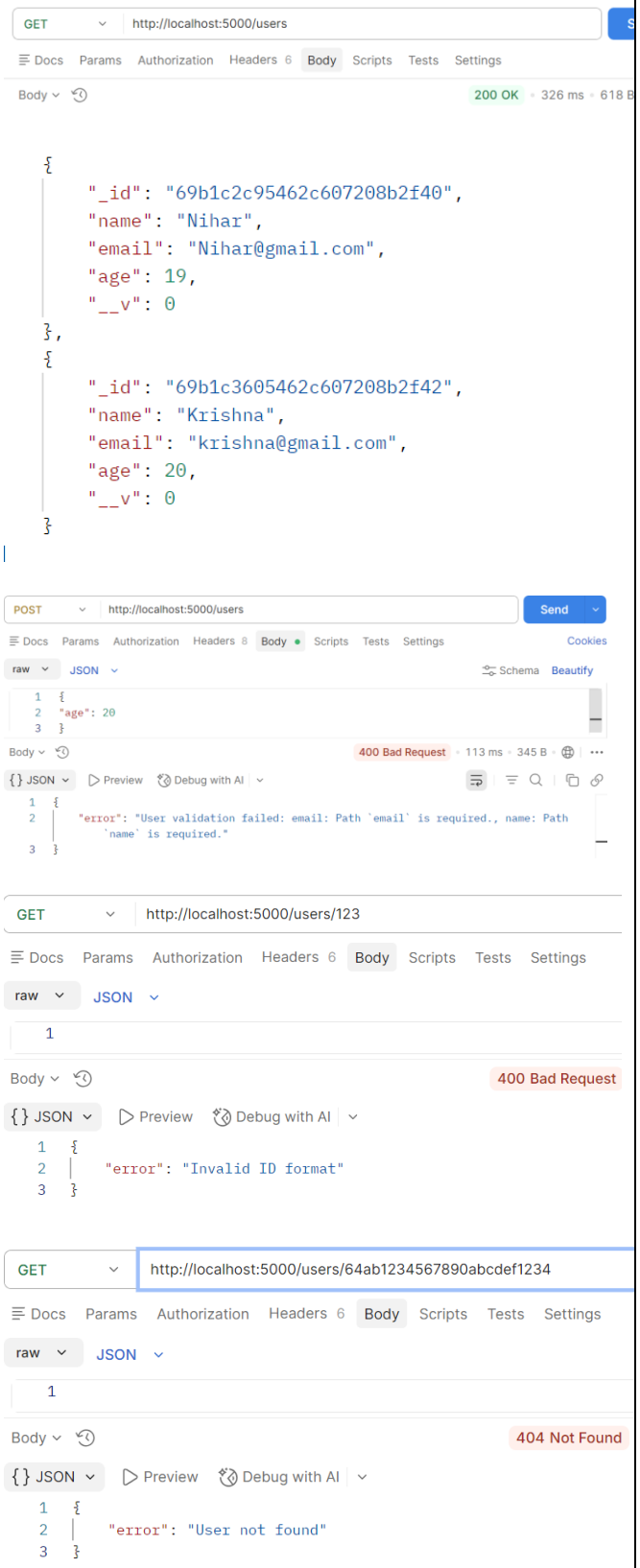
When a client sends invalid or incomplete data, the API returns a **400 Bad Request** status code. This indicates that the request cannot be processed because required fields are missing or the request format is incorrect.

---

2. What happens if MongoDB server is stopped?

If the **MongoDB** server is stopped, the backend application cannot connect to the database. As a result, database operations such as fetching or inserting data fail, and the API may return a **500 Internal Server Error** or show a database connection error.

---

3. How does proper error handling improve API reliability?

Proper error handling ensures that the application does not crash when an error occurs. It provides meaningful error messages and appropriate status codes to the client, making the API more stable, easier to debug, and more reliable for users.